Scalability of concurrent data structures

Peter Sestoft, ITU

KCS Practical Concurrent and Parallel Programming

Monday 26 October 2026

Plan

- Platform threads versus virtual threads
- Futures (or tasks) running on the executor framework
- Multi-core CPUs and cache coherence
- Case studies of multi-threaded performance
  - Threadsafe long (64-bit) counters
  - Threadsafe hash map implementations

Java virtual threads are suitable for blocking operations

- Fetching URLs one at a time, slow, mostly waiting for web server response

```
for (String url : urls)
  ... getPage(url); ...
```

Blocks thread until response received

- Fetch all in parallel, one virtual thread per URL, all waiting in parallel
  - The blocking **`getPage`** is bad for platform threads and tasks: blocks an OS thread

```
List<Thread> threads = new ArrayList<Thread>();
for (String url : urls) {
  final String myUrl = url;
  threads.add(Thread.ofVirtual().start(() ->
              { ... getPage(myUrl); ... }));
}
for (Thread t : threads)
  t.join();
```

... so copy into a final variable

Virtual threads wait in parallel

Variables captured in lambda must be final ...

TestDownloadVirtualThread.java

Java virtual threads also usable for CPU-bound work

- Example: Counting prime numbers between from = 0 and to = 99,999

```
long count = 0;
for (int i=from; i<to; i++)
  if (isPrime(i))
    count++;
```

- Divide work into chunks, e.g. 0-9999, 10000-19999, …, 90000-99999

```
final LongAdder lc = new LongAdder();
for (int t=0; t<threadCount; t++) {
  final int from = perThread * t, to = perThread * (t+1);
  threads[t] = tb.start(() -> {
    for (int i=from; i<to; i++)
      if (isPrime(i))
        lc.increment();
  });
```

Thread.Builder, e.g. `tb = Thread.ofVirtual()`

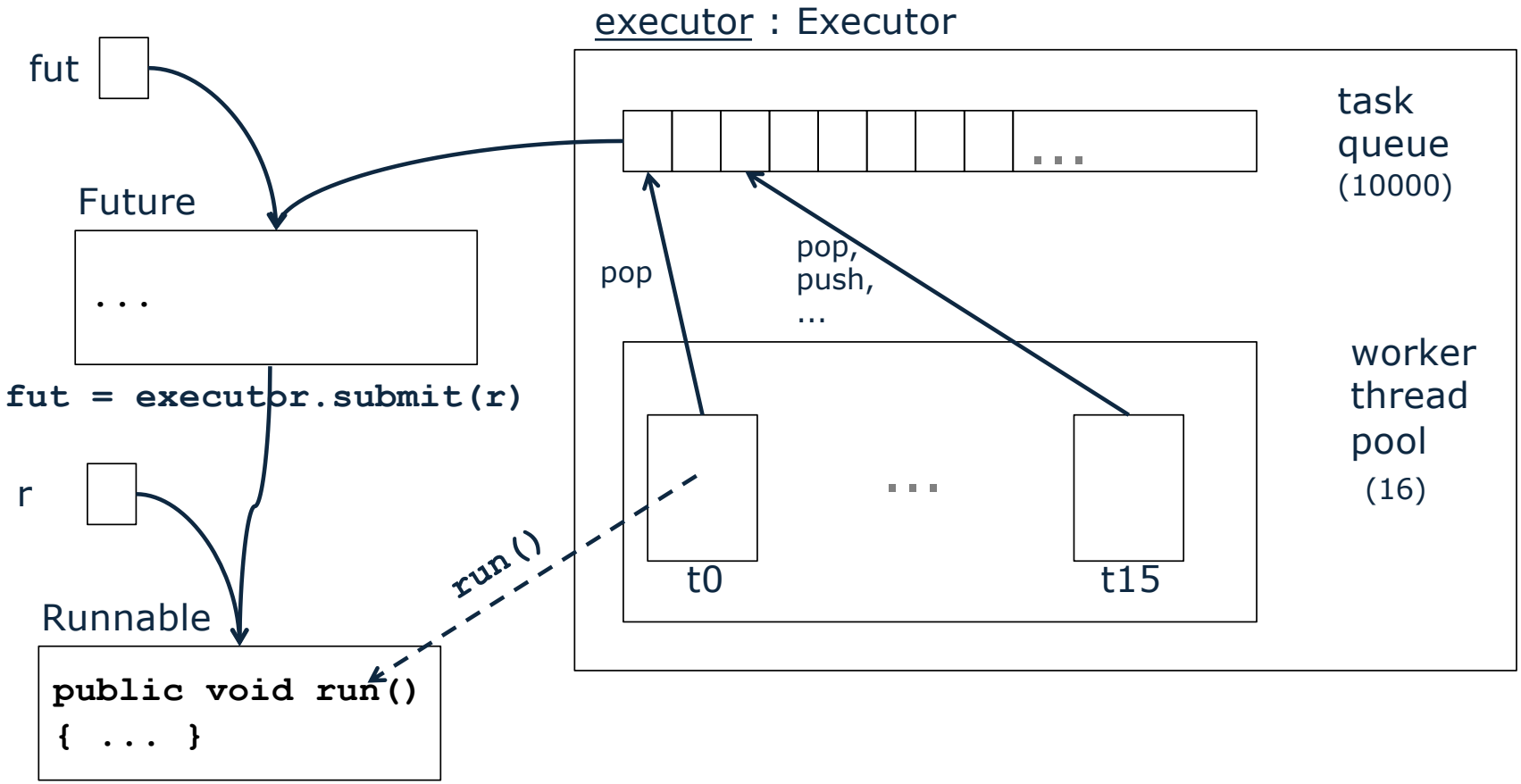
TestCountPrimesThreads.java

Java futures/tasks and executors for CPU-bound work

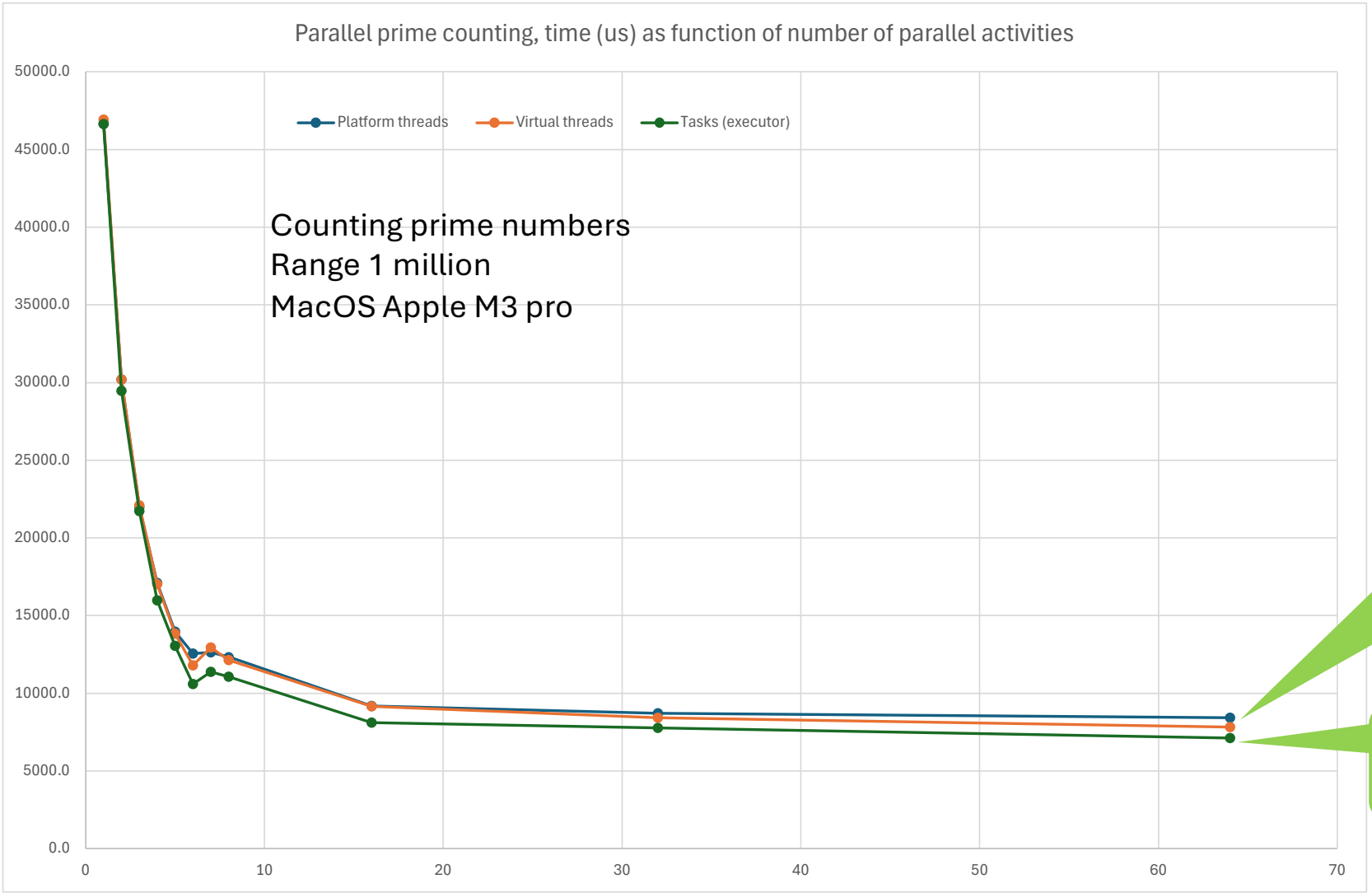
```
ExecutorService executor = Executors.newWorkStealingPool();
final LongAdder lc = new LongAdder();
List<Future<?>> futures = new ArrayList<Future<?>>();
for (int t=0; t<taskCount; t++) {
  final int from = perTask * t, to = perTask * (t+1);
  futures.add(executor.submit(() -> {
    for (int i=from; i<to; i++)
      if (isPrime(i))
        lc.increment();
  }));
}
... for (Future<?> fut : futures) fut.get(); ...
```

- Better for CPU-work but more cumbersome to use than virtual threads
- More ways to control execution

Dynamics of the executor framework



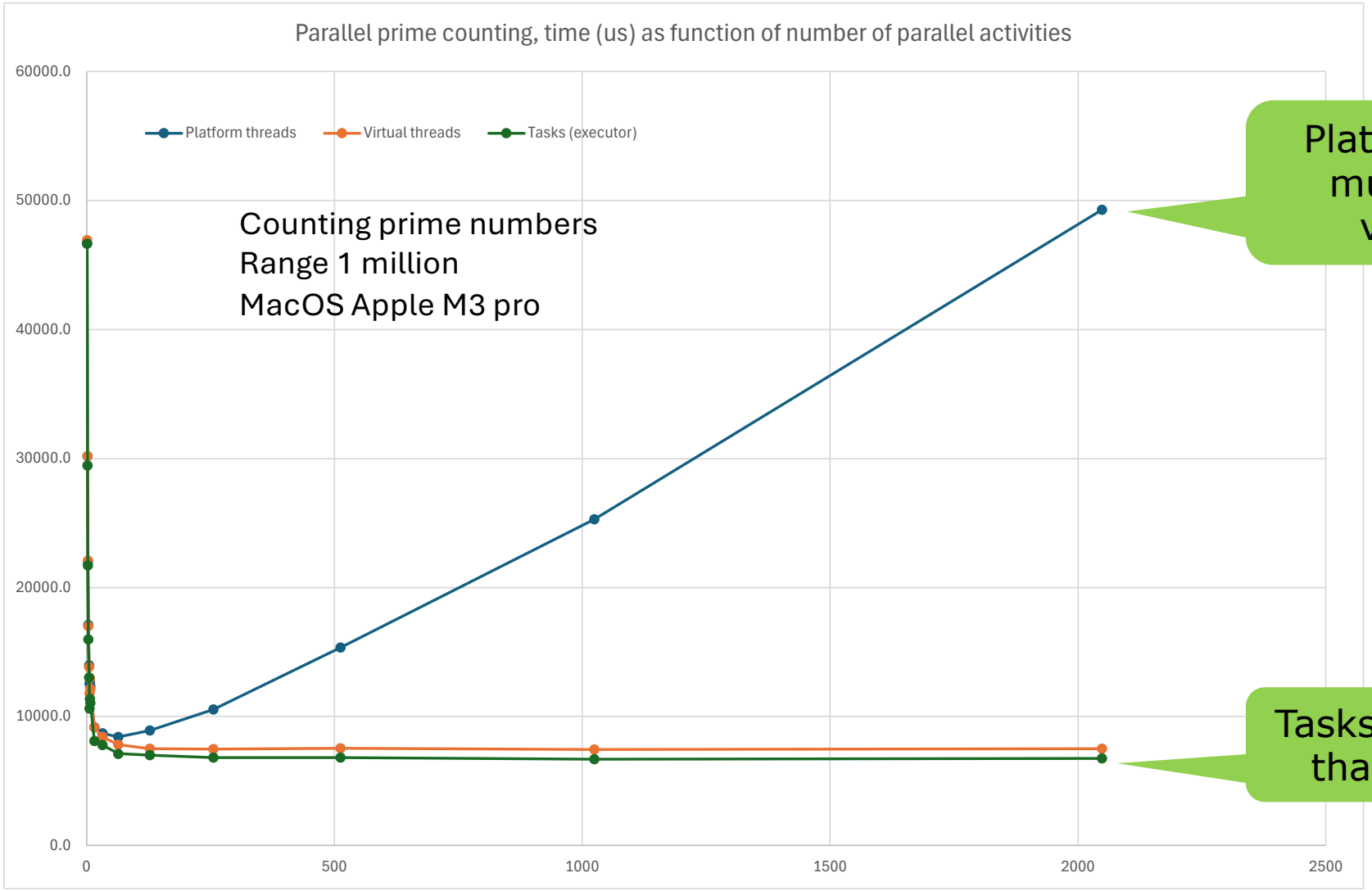
Prime counting time for few (1-64) threads/tasks



Platform threads and virtual threads are equally fast

Tasks are slightly faster than threads

Prime counting time for many (1-2048) threads/tasks



Platform threads are much slower than virtual threads

Tasks are slightly faster than virtual threads

When to use { platform, virtual } threads and tasks

Mechanism	Suitable for	Cost to create
Platform thread	All operations	High
Virtual thread	Blocking operations	Low
Task or future	Non-blocking operations	Low

- Never create many platform threads
- Do use virtual threads for blocking operations (I/O, database, …)
- Do use tasks for CPU-intensive work if performance is crucial
- Otherwise, for simplicity, use virtual threads for everything

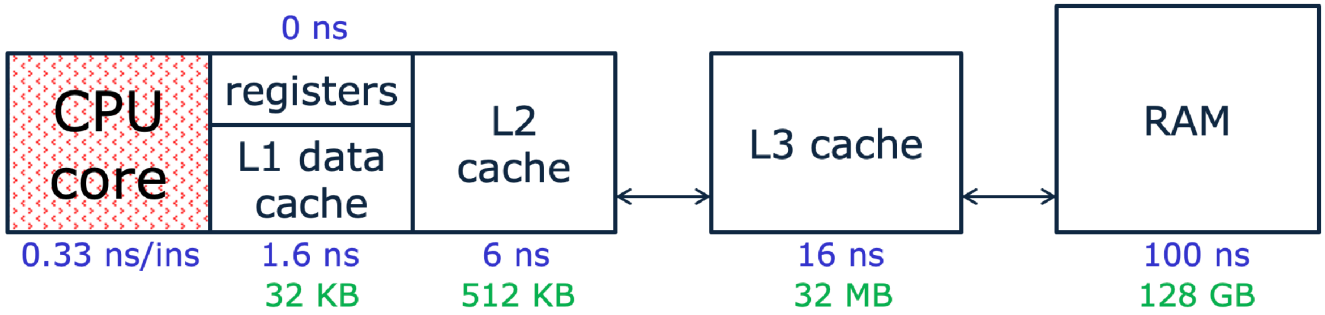
Caches: CPUs have become faster, RAM has not

1985



Arm1

2021

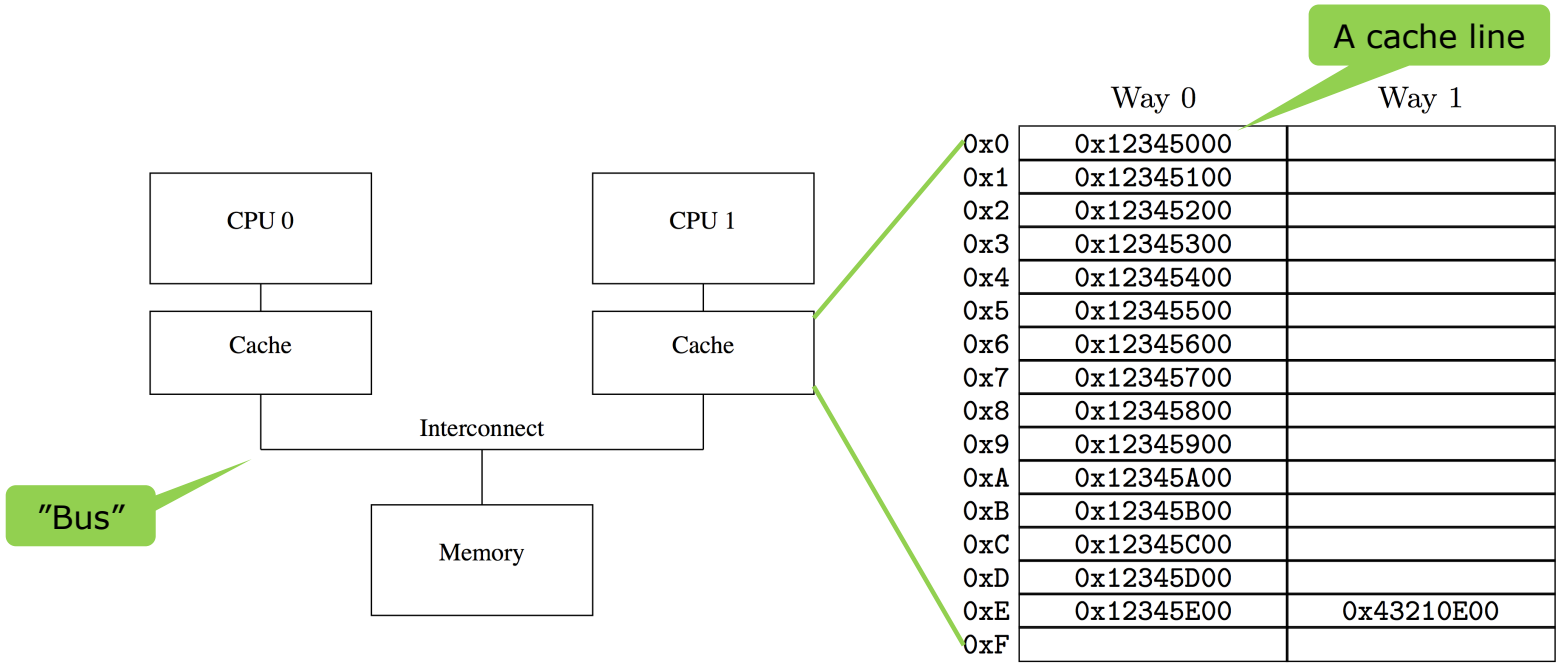


AMD EPYC 7313

- Caches bridge the CPU – RAM speed gap
- Works well when there is access locality
- Shown for a single core - what about *multiple cores*?

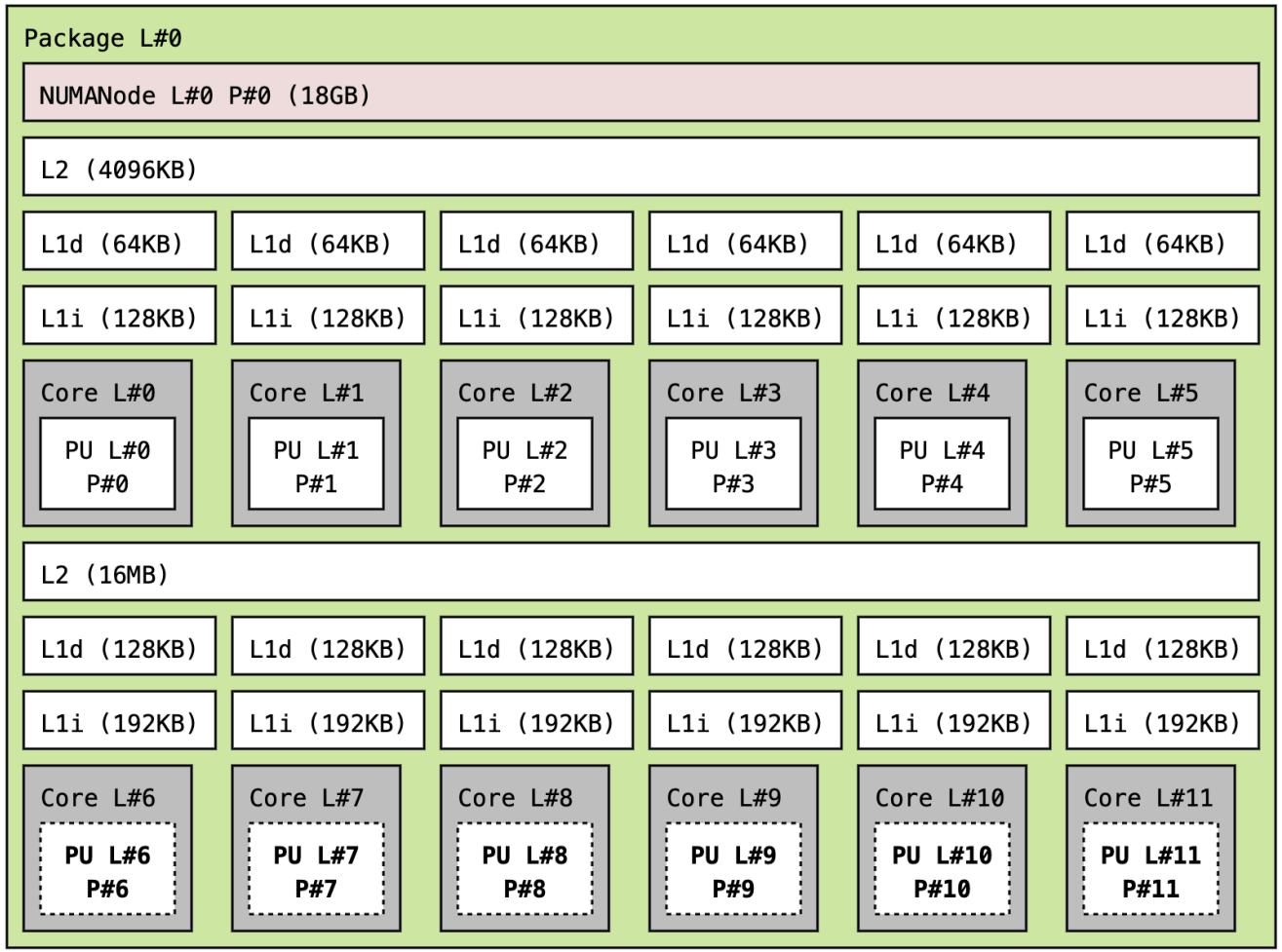
Multi-core: each core has a cache

- A cache line corresponds to a memory address
- The same memory address may be held in multiple caches



McKenney 2010: Memory barriers

How do caches agree on value of a given memory address?



By hwloc `lstopo`

Apple M3 pro
Laptop CPU, 2023
37 bn transistors

6 perf + 6 efficiency cores

- L1/core: 128KB d + 192KB i
- L2/6cores: 16MB
- L3: ?



By hwloc **`lstopo`**

AMD EPYC 7313 Zen3
Server CPU, 2021
16.6bn transistors

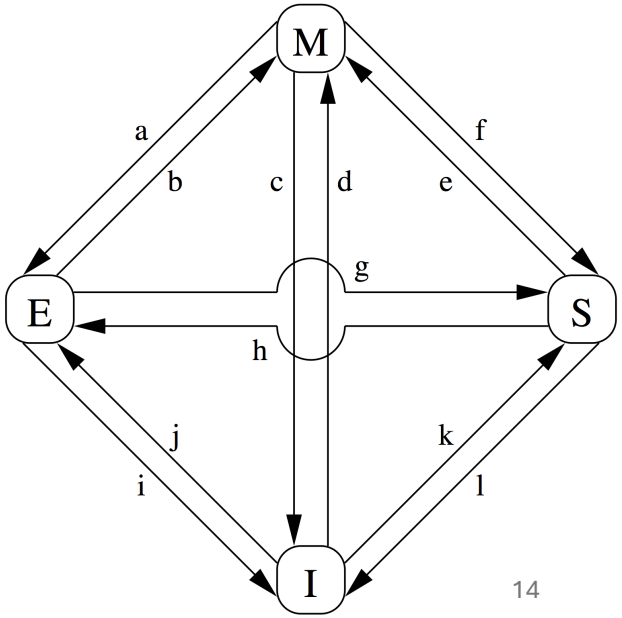
2 x 4 x 4 cores x 2 SMT

- L1/core: 32KB d + 32KB i
- L2/core: 512KB
- L3/4cores: 32 MB

Cache agreement (coherence) by MESI protocol

State	Cache line	Excl	RAM	To read	To write
Modified	Modified by me	Y	stale	from cache	to cache
Exclusive	Not modified	Y	fresh	from cache	to cache -> M
Shared	Others have it too	N	fresh	from cache	send invalidate
Invalid	Not in use by me	-	-	elsewhere	send invalidate

- A cache line
  - has 4 possible states M, E, S, I
  - and 12 transitions a-l between states
- Cache messages
  - sent by cores to others
  - via cache bus
  - to make caches agree



Transitions and messages

		Cause	I send	I receive	My response
M	a	(Send update to RAM)	writeback	-	-
E	b	Write	-	-	-
M	c	Other wants to write	-	read inv	read resp, inv ack
I	d	Atomic read-mod-write	read inv	read resp, inv ack*	-
S	e	Atomic read-mod-write	read inv	inv ack*	-
M	f	Other wants to read	-	read	read resp
E	g	Other wants to read	-	read	read resp
S	h	Will soon write	inv	inv ack*	-
E	i	Other wants atomic rw	-	read inv	read resp, inv ack
I	j	Want to write	read inv	read resp, inv ack*	-
I	k	Want to read	read	read resp	-
S	l	Other wants to write	-	inv	inv ack

A write in non-exclusive state needs acknowledge ack* from *all* cores

Fast and slow cache cases

- The cache is **fast** when
  - the local core “owns” the data (state M or E), or
  - data is shared (S) but local core *only reads* it
- The cache is **slow** when
  - the data is shared (S) and we want to write it, or
  - the data is not in cache (I), possibly because “owned” by another core

Unshared mutable data

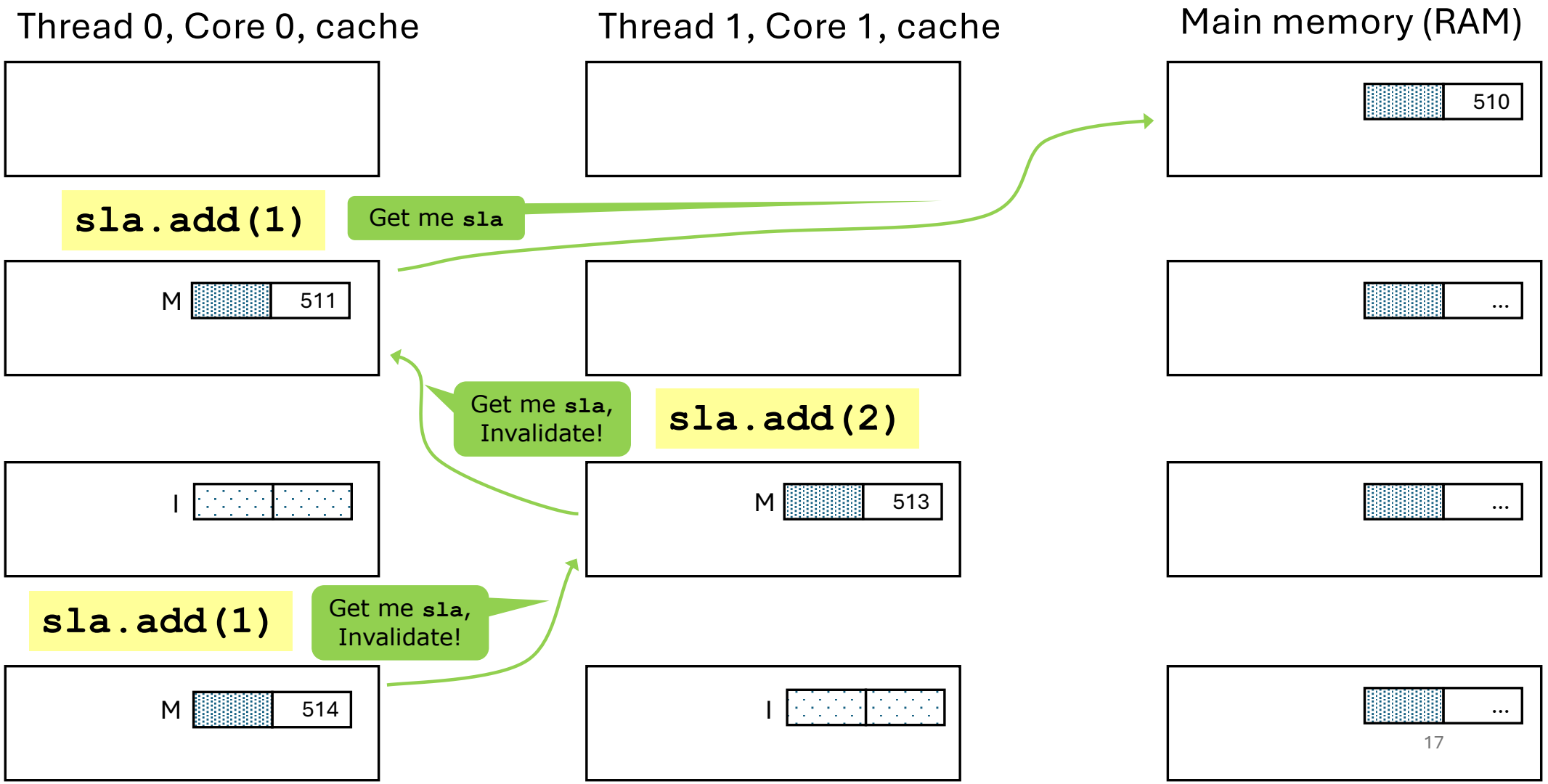
Shared immutable data

Shared mutable data

		This core wants to	Messages	Speed
M	M	Read cache line	0	fast
M	M	Write cache line	0	fast
E	E	Read cache line	0	fast
E	M	Write cache line	0	fast
S	S	Read cache line	0	fast
I	S	Read cache line	1+1	slow
S	M	Write cache line	1+N	very slow
I	M	Write cache line	1+1+N	very slow

N cores

Frequent cross-thread updates can be slow



Updates should be thread-local if possible

- **Bad** scenario: Multiple threads often *update* shared data
  - Caches fight to get the data in their own cache, in M state; put others in I state
  - “Cache ping pong” or “cache thrashing” with lots of Invalidate messages
- **Good** scenario: Thread mostly *read* shared data, *update* only own data
  - Immutable shared data stays in S state in all caches – fast
  - Thread-local mutable data stays in M state locally – fast

- NB 1: Taking a lock is itself a memory update
  - So multiple threads frequently taking the same lock is likely to be slow
- NB 2: A cache line (64 bytes) may contain multiple data items
  - So if any data item is updated, the others get invalidated in other caches
  - One may try to put independently updated data on different cache lines

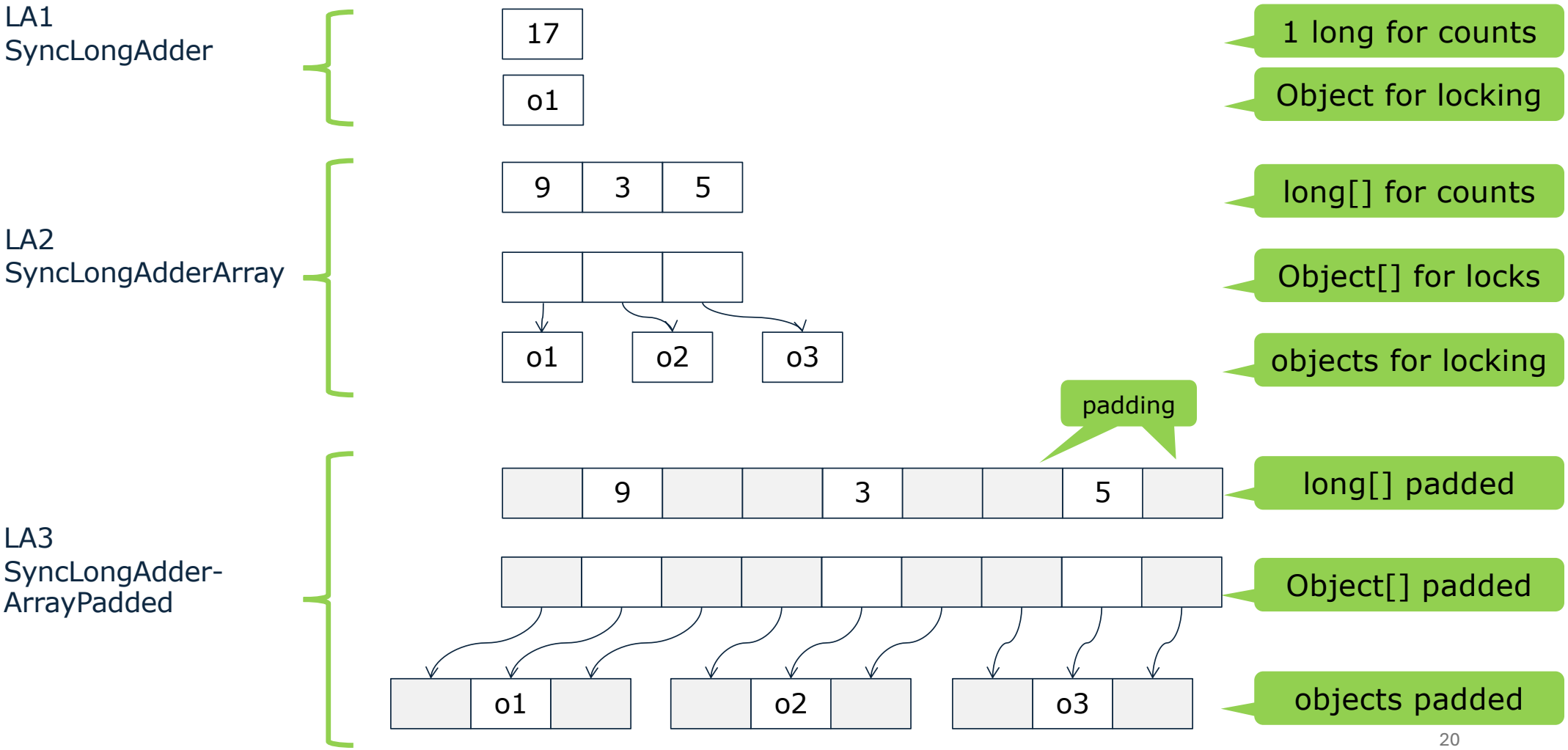
”False sharing”

Threadsafe 64-bit integer (`long`) counters in Java

```
count: 81647435391910657 99
```

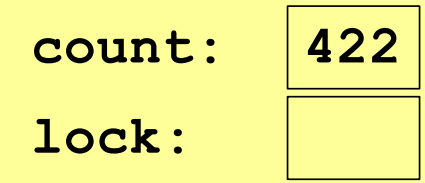
1. SyncLongAdder: Single lock protecting a **`long`** field
2. SyncLongArrayAdder: 64 locks for 64 **`longs`**; indexed by threadId % 64
3. SyncLongArrayAdderPadded: same, but with padding
4. WrappedAtomicLong: A single AtomicLong field
5. MyAtomicLong: An atomic long implemented with compare-and-set
6. ThreadLocalAtomicLong: An atomic long per thread
7. LongAdderArray: AtomicLongArray, indexed by threadId % 64
8. LongAdderArrayPadded: same, but with padding
9. WrappedLongAdder: a java.util.concurrent.atomic.LongAdder

Memory layouts of LA1, LA2 and LA3



LA1. SyncLongAdder

```
class SyncLongAdder implements ILongAdder {
  private long count = 0;
  private final Object lock = new Object();
  public void add(long delta) {
    synchronized (lock) {
      count += delta;
    }
  }
  public long longValue() {
    synchronized (lock) {
      return count;
    }
  }
}
```



LongAdderPerformance.java

LA2. SyncLongArrayAdder – “lock striping”

```
class SyncLongArrayAdder implements ILongAdder {
  private final static int logS = 6, stripeCount = 1 << logS,
    stripeMask = stripeCount - 1;
  private final long[] counts = new long[stripeCount];
  private final Object[] locks = new Object[stripeCount];

  public SyncLongArrayAdder() {
    for (int s=0; s<stripeCount; s++)
      locks[s] = new Object();
  }

  public void add(long delta) {
    int s = (int)Thread.currentThread().threadId() & stripeMask;
    synchronized (locks[s]) {
      counts[s] += delta;
    }
  }
}
```

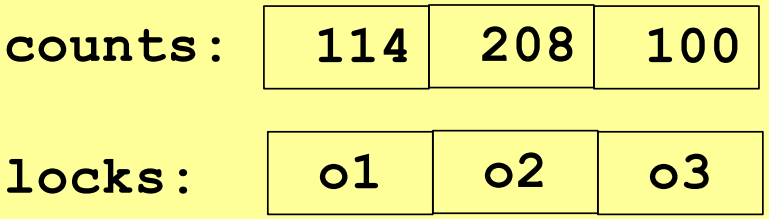
64

63 = 0b111111

Stripe

Same as: `%64`

High chance that different threads take different locks



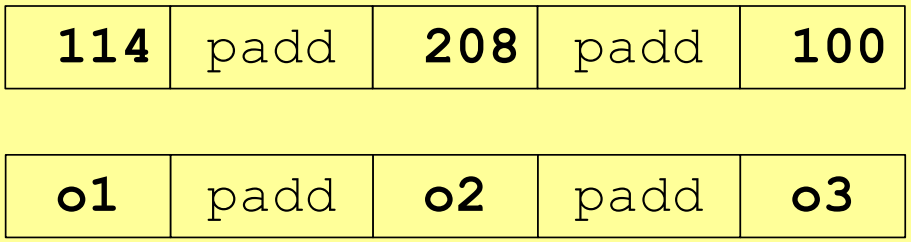
LA3. SyncLongArrayAdderPadded – add padding

```
class SyncLongArrayAdderPadded implements ILongAdder {
  private final static int logS = 6, stripeCount = 1 << logS,
    stripeMask = stripeCount - 1, padding = 16;
  private final long[] counts = new long[stripeCount * padding];
  private final Object[] locks = new Object[stripeCount * padding];

  public SyncLongArrayAdderPadded() {
    for (int s=0; s<stripeCount * padding; s++)
      locks[s] = new Object();
  }

  public void add(long delta) {
    int s = (int)Thread.currentThread().threadId() & stripeMask,
        p = s * padding;
    synchronized (locks[p]) {
      counts[p] += delta;
    }
  }
}
```

Padding puts the counts and the locks on different cache lines



LA4. WrappedAtomicLong

```
class WrappedAtomicLong implements ILongAdder {
  private final AtomicLong atomicLong = new AtomicLong();

  public void add(long delta) {
    atomicLong.addAndGet(delta);
  }

  public long longValue() {
    return atomicLong.get();
  }
}
```

From Java library
java.util.concurrent.atomic

LA5. ThreadLocalAtomicLong

```
class ThreadLocalAtomicLong implements ILongAdder {
  private final ConcurrentHashMap<AtomicLong,Boolean> allAtomicLongs =
    new ConcurrentHashMap<>();
  private final ThreadLocal<AtomicLong> atomicLongHolder =
    ThreadLocal.withInitial(() -> {
      AtomicLong al = new AtomicLong();
      allAtomicLongs.put(al, Boolean.TRUE);
      return al;
      });

  public void add(long delta) {
    atomicLongHolder.get().addAndGet(delta);
  }

  public long longValue() {
    return allAtomicLongs.keySet().stream().mapToLong(al -> al.get()).sum();
  }
}
```

From Java library java.lang

Allocates one AtomicLong per thead

Bad when there are very many threads

LA6. MyAtomicLong

```
class MyAtomicLong implements ILongAdder {
  private final AtomicLong atomicLong = new AtomicLong();

  public void add(long delta) {
    long oldValue, newValue;
    do {
      oldValue = atomicLong.get();
      newValue = oldValue + delta;
    } while (!atomicLong.compareAndSet(oldValue, newValue));
  }

  public long longValue() {
    return atomicLong.get();
  }
}
```

LA7. LongAdderArray

```
class LongAdderArray implements ILongAdder {
  private final static int logS = 6, stripeCount = 1 << logS,
    stripeMask = stripeCount - 1;
  private final AtomicLongArray counters
    = new AtomicLongArray(stripeCount);

  public void add(long delta) {
    counters.addAndGet(
      ((int)Thread.currentThread().threadId() & stripeMask),
       delta);
  }
}
```

Like LA2 SyncLongArrayAdder, but using CAS-based atomics, no locks

LA8. LongAdderArrayPadded

```
class LongAdderArrayPadded implements ILongAdder {
  private final static int logS = 6, stripeCount = 1 << logS,
    stripeMask = stripeCount - 1, padding = 16;
  private final AtomicLongArray counters
    = new AtomicLongArray(stripeCount * padding);

  public void add(long delta) {
    counters.addAndGet(
      ((int)Thread.currentThread().threadId() & stripeMask) * padding,
       delta);
  }
}
```

Like LA3 SyncLongArrayAdderPadded, using padding to scatter the AtomicLong objects

LA9. WrappedLongAdder

```
class WrappedLongAdder implements ILongAdder {
  private final LongAdder longAdder = new LongAdder();
  public void add(long delta) {
    longAdder.add(delta);
  }
  public long longValue() {
    return longAdder.longValue();
  }
}
```

The official Java scalable multithreaded counting

Workload for long adders

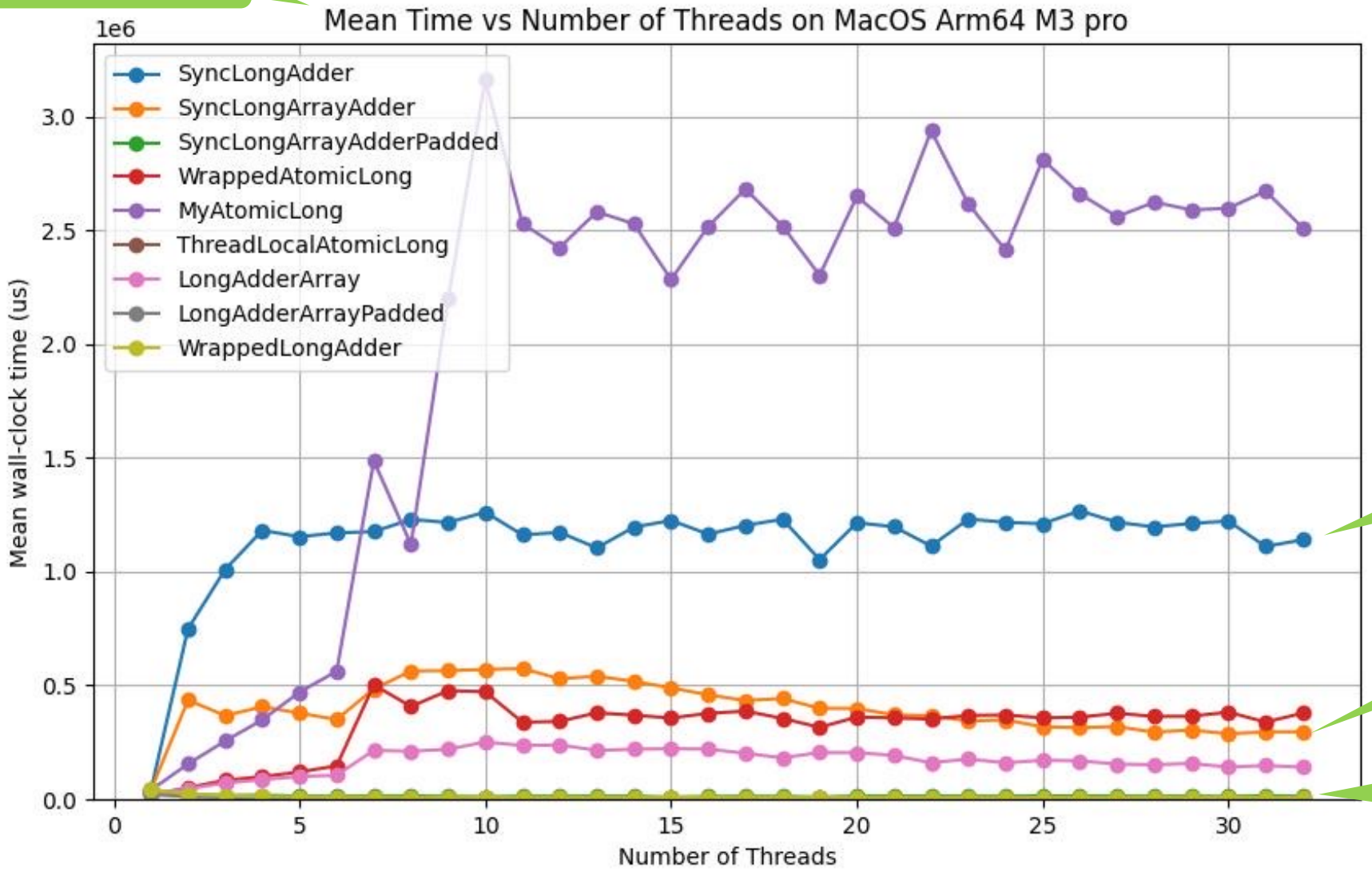
- N threads simultaneous do:
  - Add i = 1, 2, …, K to the long counter
- Thread count N = 1, 2, …, 32 and K = 10,000,000 / N

- We measure wall-clock time for all threads to finish

- A sign of good scalability:
  - Wall-clock time decreases with increasing thread count N
  - … when N is less than the number of cores
- A sign of poor scalability:
  - Wall-clock time stays constant or increases with increasing thread count N

Lower is better

Longadders, laptop **Apple M3 pro**, all 9

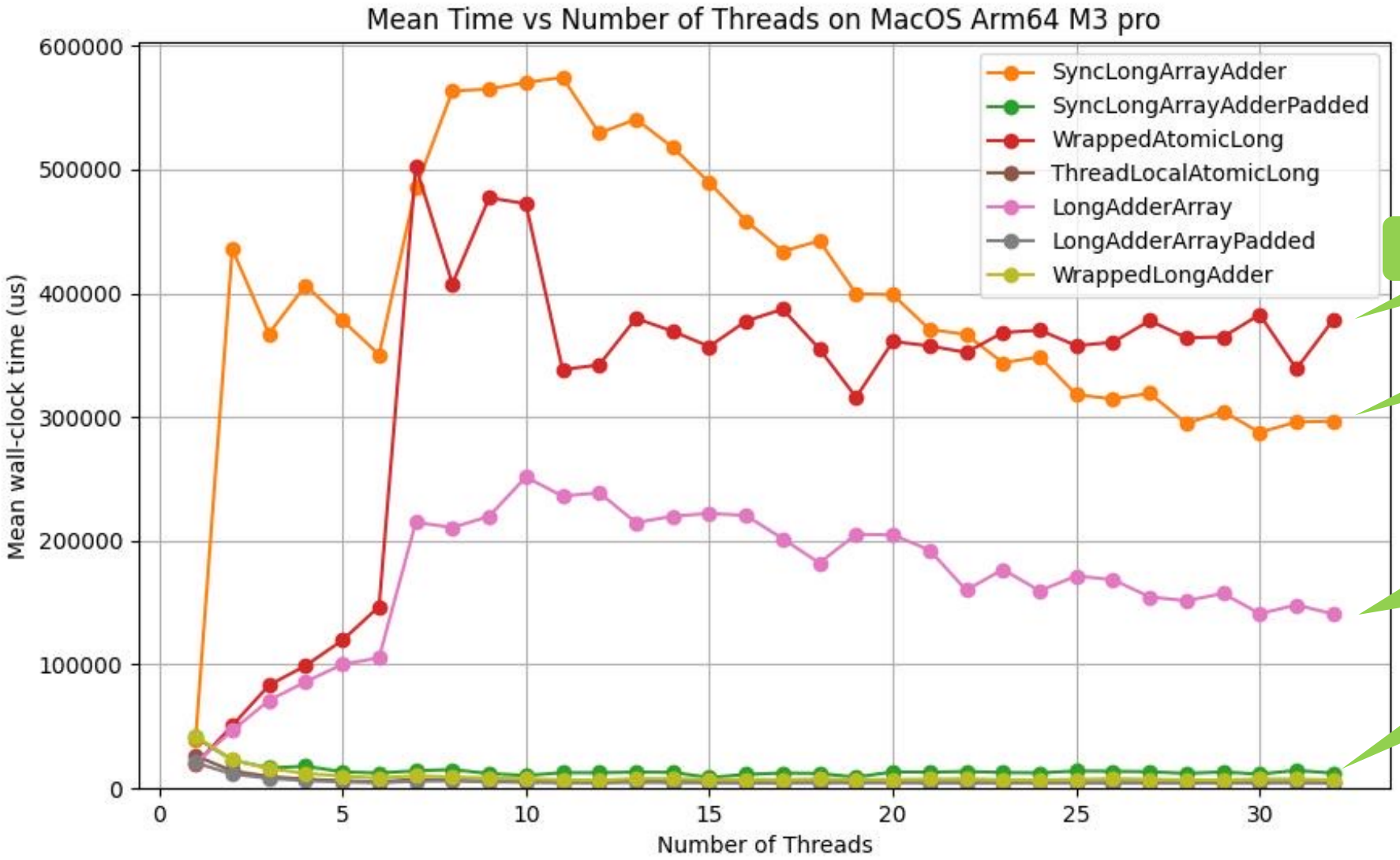


1 lock

64 locks

64 locks, padding

Longadders, laptop **Apple M3 pro**, 7 best



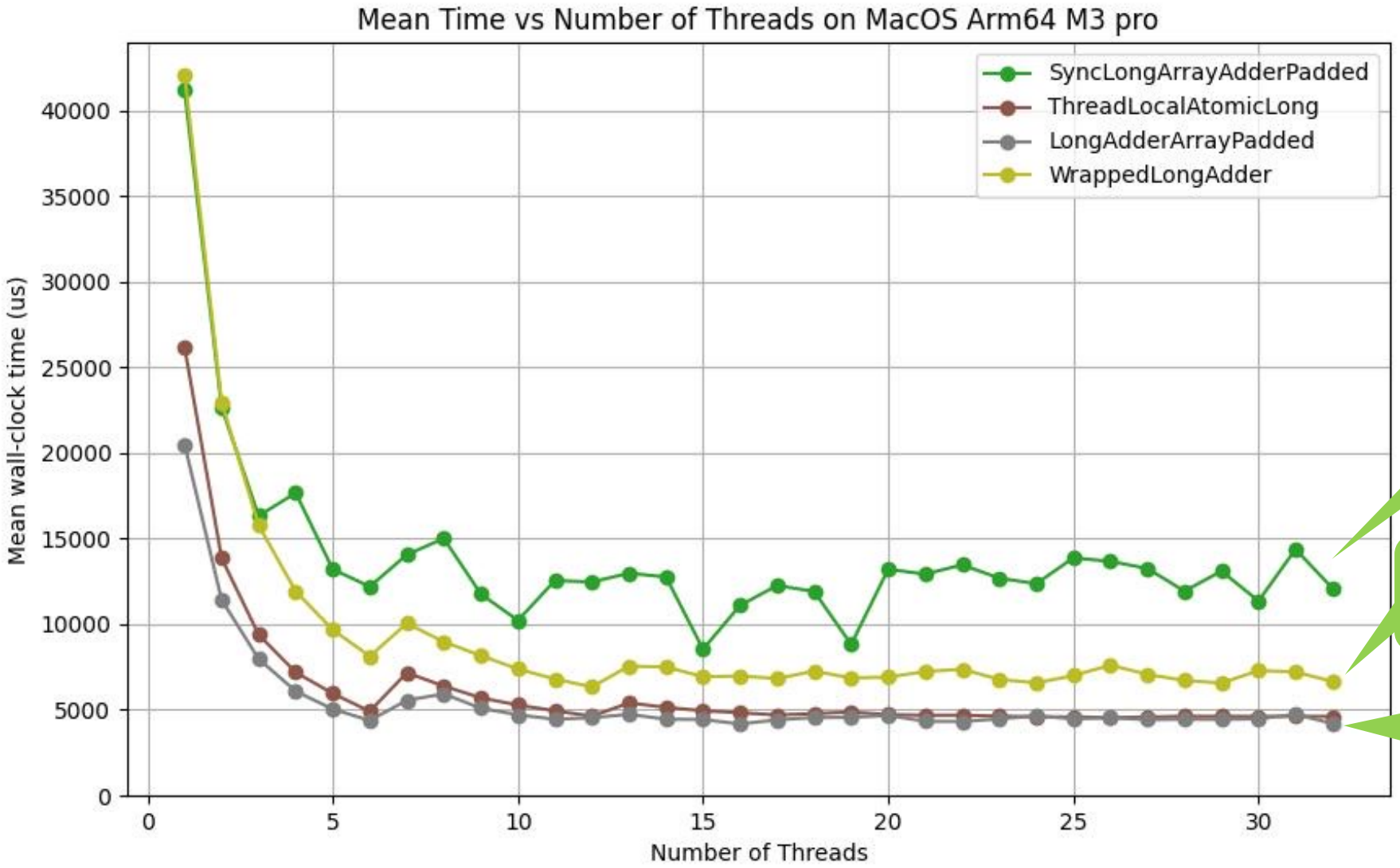
AtomicLong

64 locks

AtomicLongArray

64 locks, padding

Longadders, laptop **Apple M3 pro**, 4 best

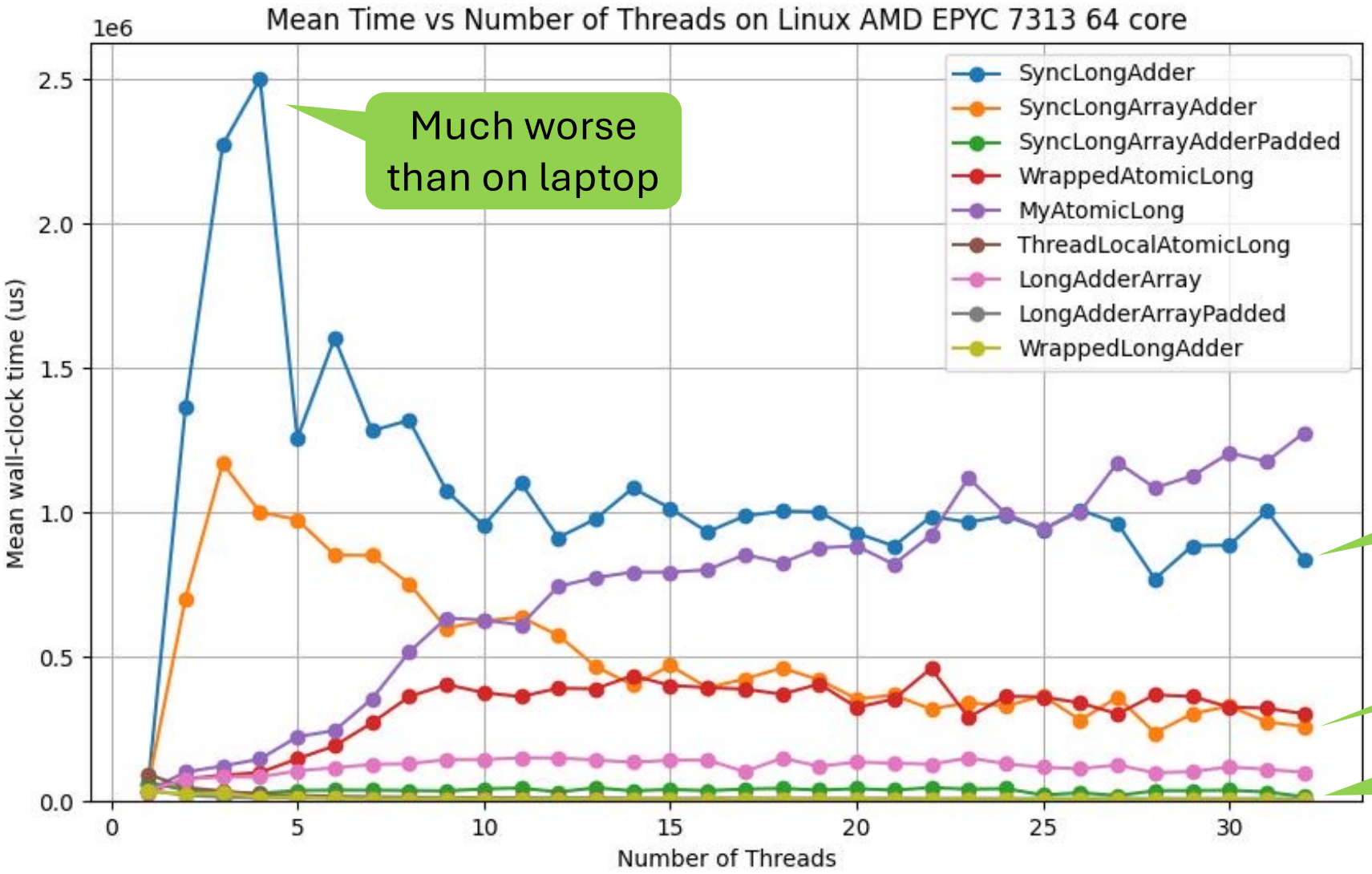


64 locks, padding

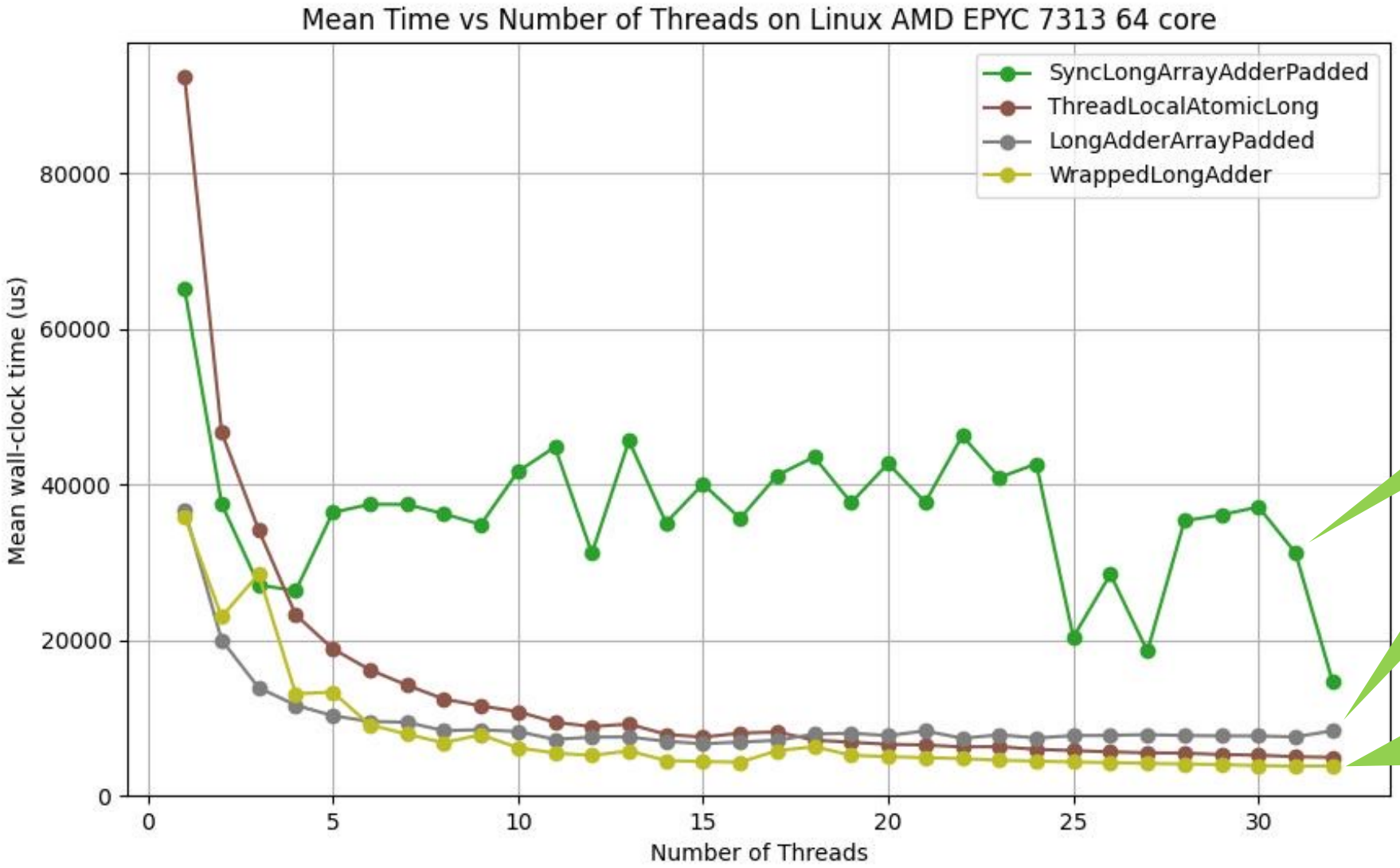
Java's LongAdder

AtomicLongArray, padded

Longadders, server **AMD EPYC 7313**, all 9



Longadders, server **AMD EPYC 7313**, 4 best



Slower than on M3 pro

64 locks, padding

AtomicLongArray, padded

Java's LongAdder

Conclusion from LongAdder case study

- Using locks (Java synchronized) is not necessarily slow
  - But a single lock leads to contention **and** to serious cache coherence overhead
- Lock striping can speed up concurrent code
- It is all-important to avoid contention for memory cache lines
  - Distinct locks on **same** cache line lead to serious cache coherence overhead
  - Use “padding” of arrays to spread locks and longs on **different** cache lines
- The java.util.concurrent.atomic.LongAdder does this correctly

- These results hold for five different CPU architectures 2014-2023

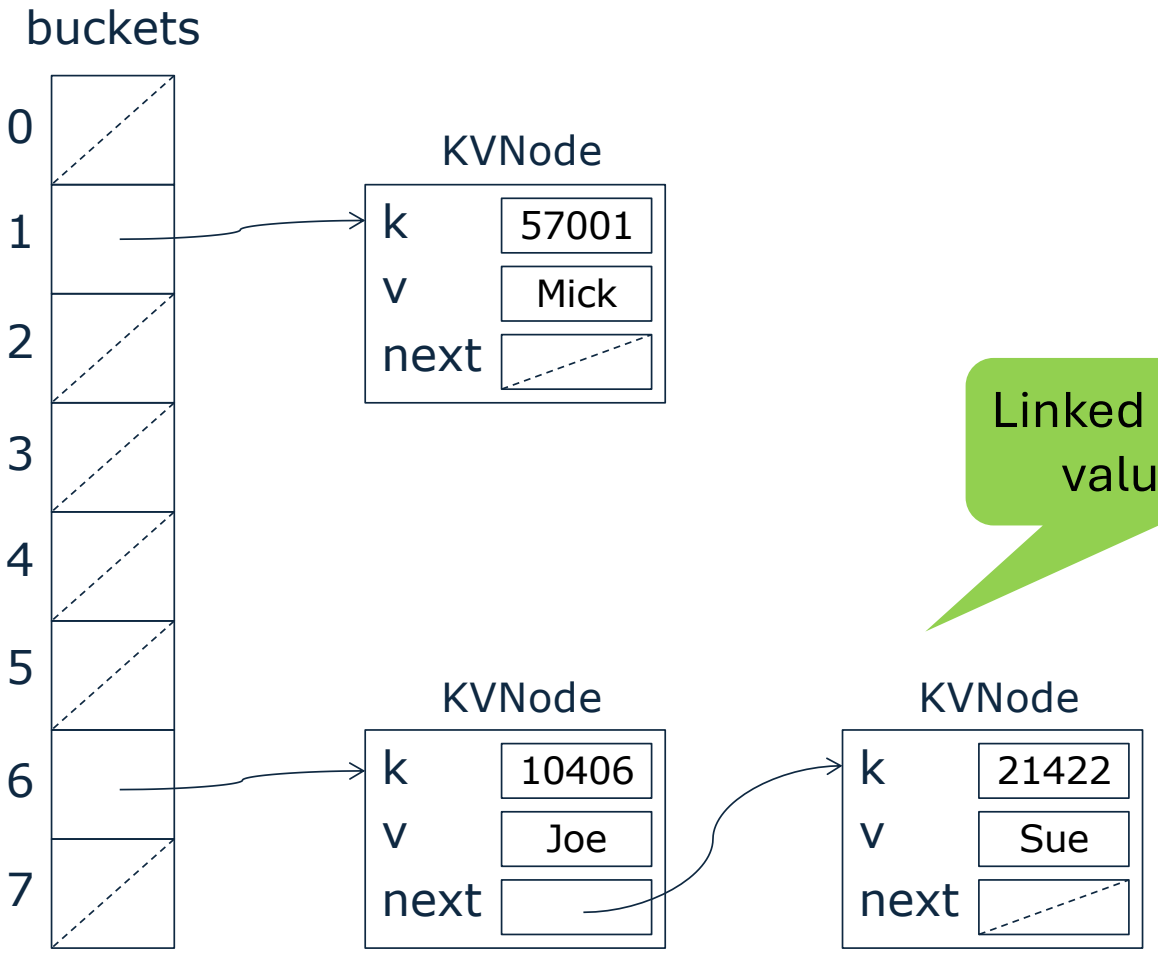
Case study 2: Threadsafe hash maps in Java

1. SynchronizedMap: Single lock protecting a hash map
2. StripedMap: 16 locks protecting separate parts of the hash map
3. StripedMapPadded: 16 locks, with padding between lock objects
4. StripedWriteMap: lock-striped map with non-blocking reads
5. StripedWriteMapPadded: same, with padding between lock objects
6. StripedLevelWriteMap: lock-striped via two-level array, non-blk reads
7. HashTrieMap: Lock-free hash array mapped trie (Prokopec, Bagwell)
8. WrapConcHashMap: the java.util.concurrent.ConcurrentHashMap

HM1. SynchronizedMap

A single lock protecting a standard sequential hash map

Buckets array, indexed by hashcode % 8



Linked list of (key, value) pairs

SynchronizedMap.java

HM2. StripedMap

- Same as before, but now using lock striping with 32 locks
- Create an array of 32 lock Objects, use their intrinsic locks
- Each lock protects every 32th bucket array entry and its linked list
- Two threads accessing different stripes can run concurrently

StripedMap.java

HM4. StripedWriteMap

- Same as before, but now making read access non-blocking
- Make the linked lists of (key, value) pairs immutable
- Then read access (**`get`**, ...) can run concurrently with updates
- Good scalability if hash map reads are more frequent than writes
- Writes duplicate (key, value) pairs, so needs garbage collector
  - This strategy is not usable in C or C++ or Rust for lack of GC
  - But some Rust concurrent hashmap have their own garbage collector...

StripedWriteMap.java

HM5. StripedWriteMapPadded

- Same as before, but add padding to the locks array
- Create an array of 32 lock Objects
- But put 15 unused Objects between every two usable lock Objects
- Hopefully the lock Objects will be on different cache lines

StripedWriteMapPadded.java

HM6. StripedLevelWriteMap

- Lock striping, but in a different way
- Instead of a single bucket array **`KVPair[]`**, make a two-level structure:
  - Top array **`KVPair[][]`** of references to stripe-specific bucket arrays **`KVPair[]`**
  - Each lock protects one of the subbucket arrays
- Subbucket arrays can be resized independently and concurrently
- The extra indirection costs some access time
- If one is careful with allocation order, no padding is needed

StripedLevelWriteMap.java

HM7. HashTrieMap

Lock-free CAS-based operations (Prokopec et al 2011, 2017)

Based on hash-array mapped tries (Bagwell 2000), Florent Le Gall's code

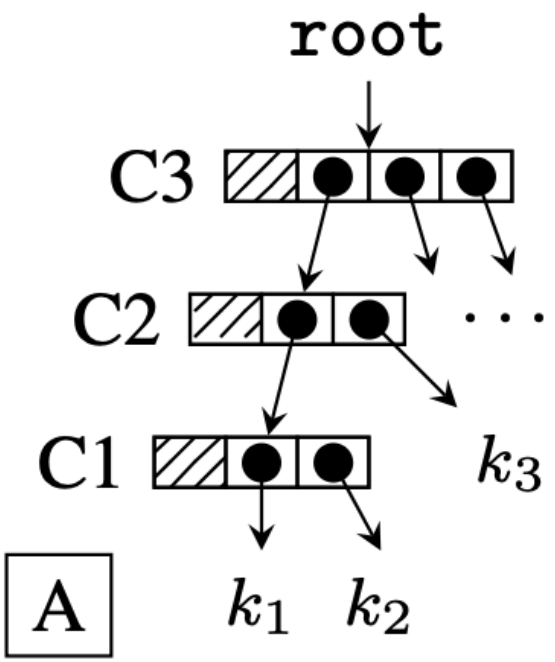
Real-time friendly, shrinks memory use, can be made persistent

Hash value is broken into blocks of bits

Block 1 = index at tree level 1, etc

Compressed storage of subtree arrays

Indirection nodes support CAS-ops



HashTrieMap.java

HM8. WrapConcHashMap

- Java's java.util.concurrent.ConcurrentHashMap, by Doug Lea et al
- Considerably reengineered over Java 6, 7, and 8; stable since Java 8
- Subtle combination of non-blocking CAS-based and lock-based
- Uses padding and other cache-aware tricks
- Ca 6400 lines of code and comments

WrapConcHashMap.java

Workload

- N threads simultaneously do K operations:

HashMapPerformance.java

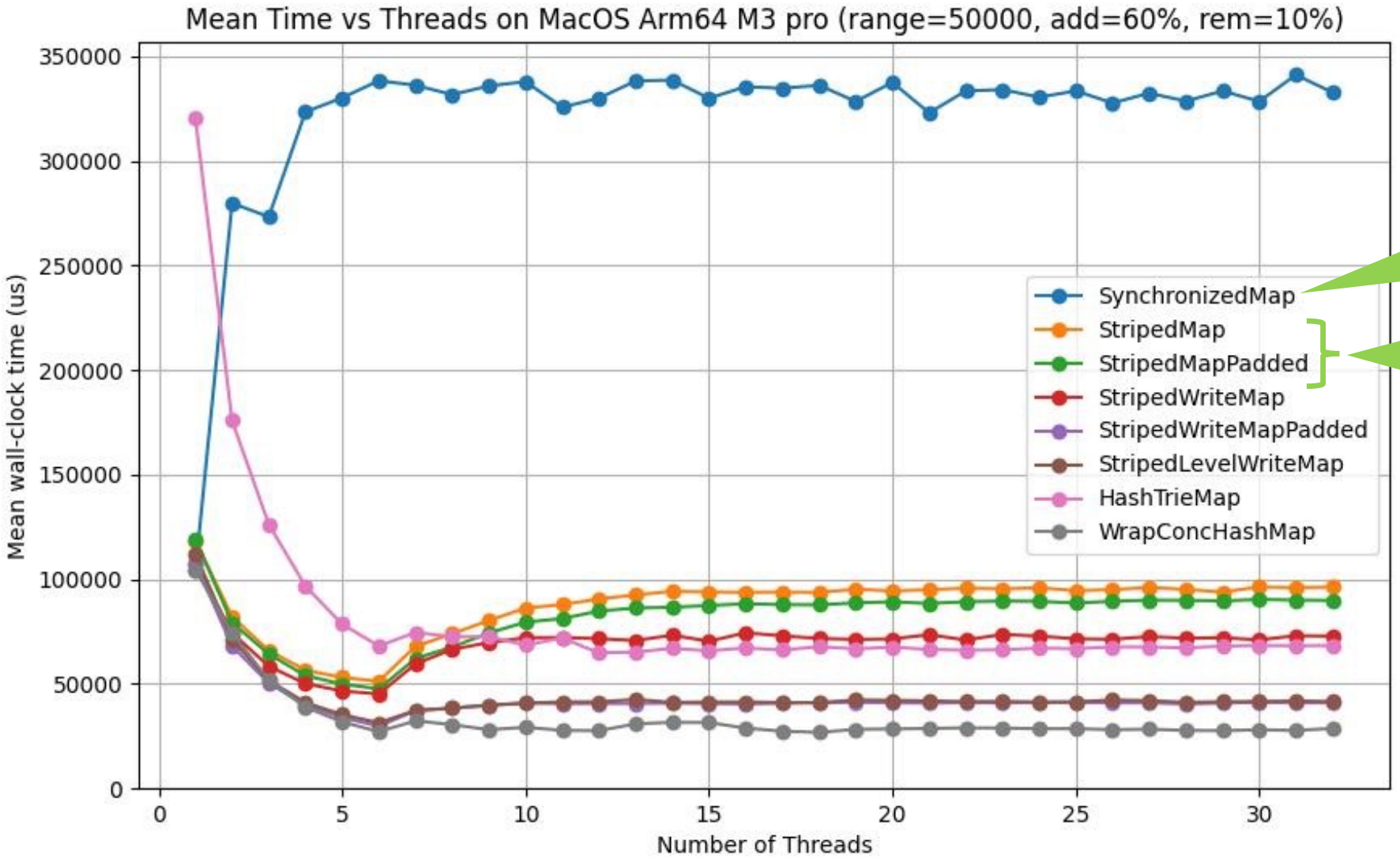
```
Integer key = random.nextInt(range);
if (!map.containsKey(key)) {
  if (random.nextDouble() < addProb)
    map.put(key, Integer.toString(key));
} else if (random.nextDouble() < removeProb)
    map.remove(key);
```

With probability addProb, add absent key

With probability removeProb, remove present key

- Thread count N = 1, 2, …, 32 and K = 5,000,000 / N
- Adjusting addProb and removeProb gives different workloads:
  - Read-dominated (60% and 2%) or write-dominated (90% and 30%) or …
- We measure wall-clock time for all threads to finish

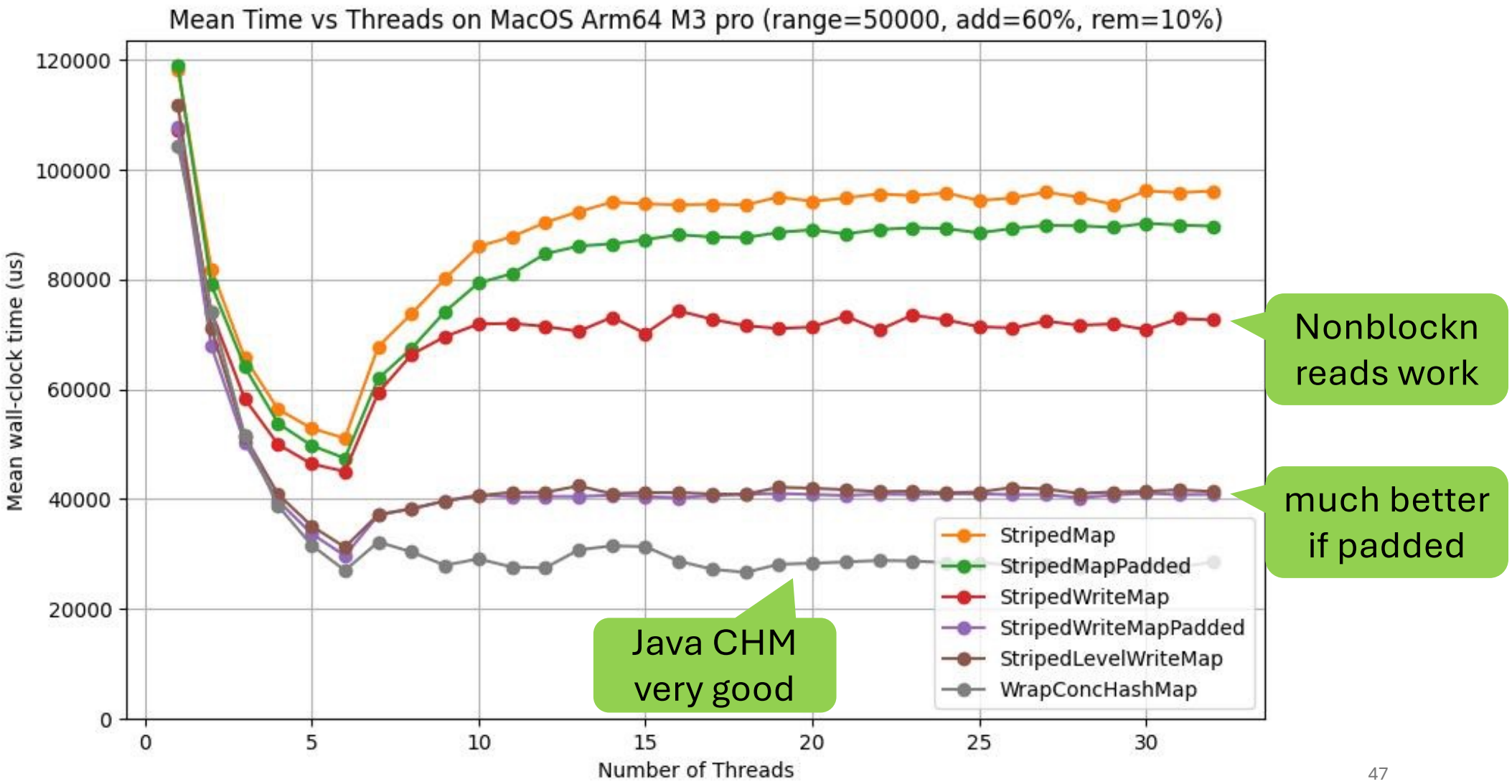
Hashmaps, read/write balanced, laptop **Apple M3 pro**, all 8



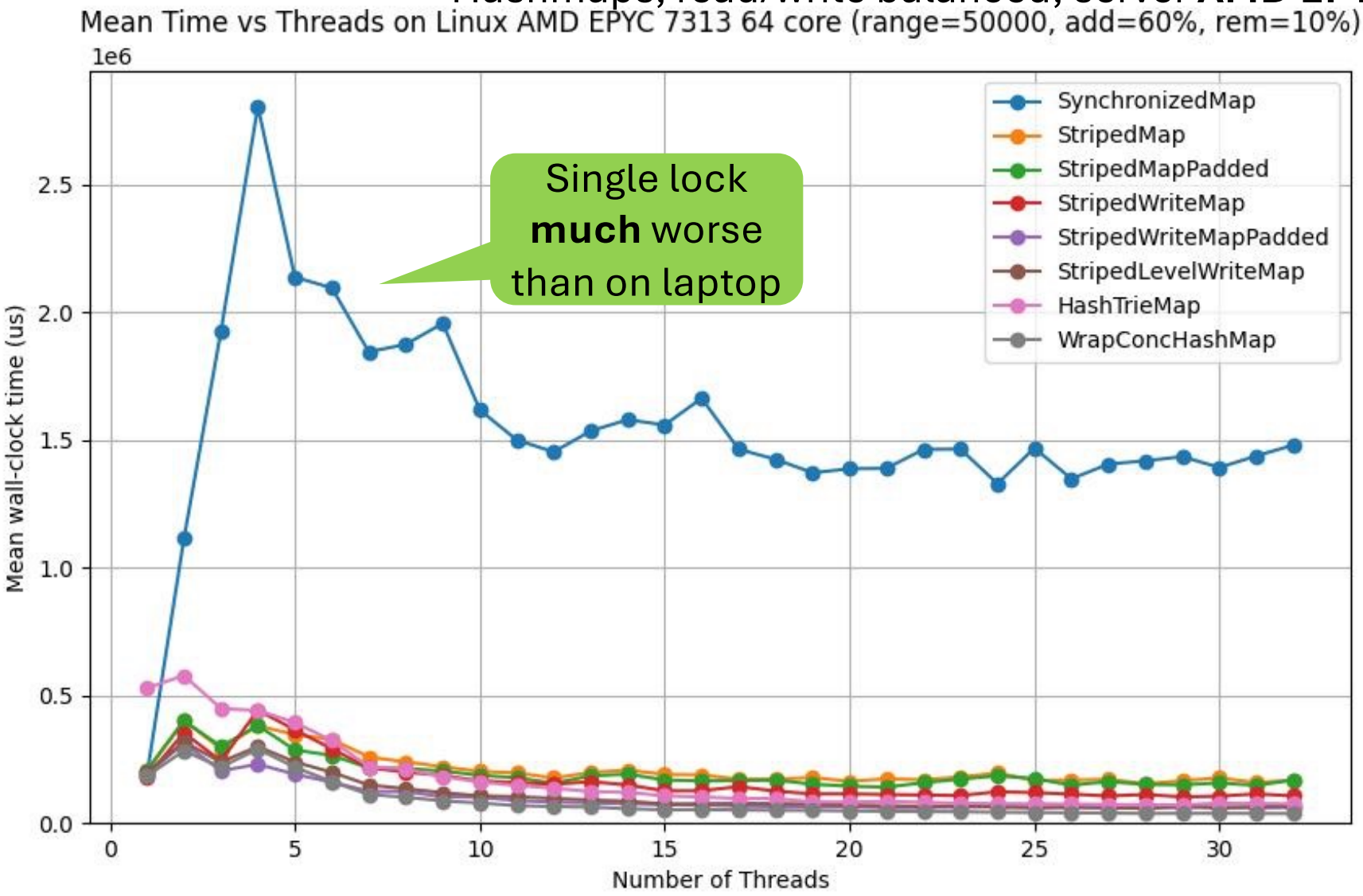
Never scalable

Also not scalable

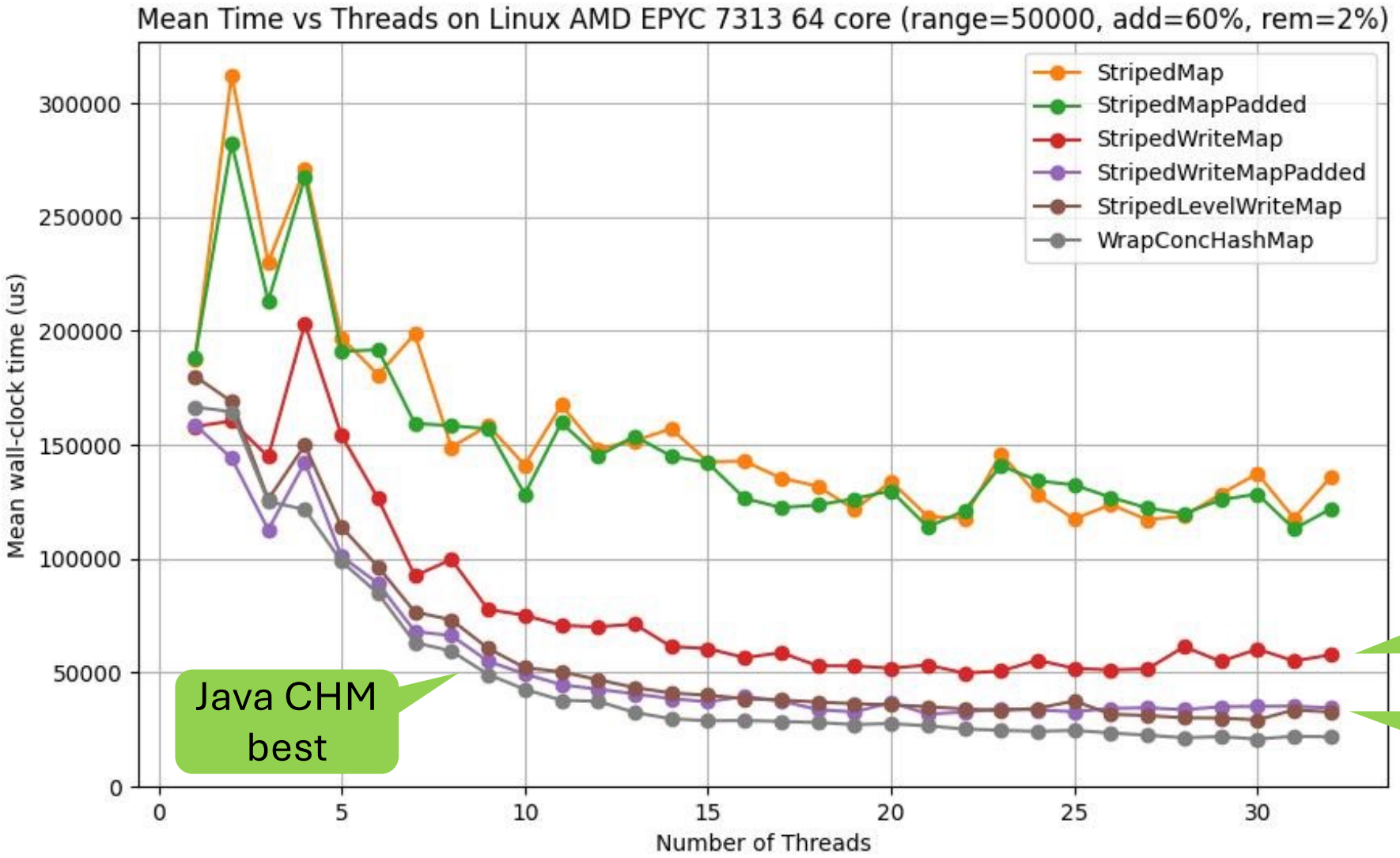
Hashmaps, read/write balanced, laptop **Apple M3 pro**, 6 best



Hashmaps, read/write balanced, server **AMD EPYC 7313**, all 8



Hashmaps, **read-dominated**, server **AMD EPYC 7313**, 6 best



Java CHM best

Nonblock works

Padding works

Learnings

- Locks can be fast: a parked thread does not waste CPU cycles
- Multiple locks – lock striping – can be even faster
  - Provided that mutable data, incl. lock objects, are **mostly thread-local**
  - So that data are core-local and hence cache-local and can stay in M state
- Hence the importance of "padding" or Java's **`@Contended`** annotation
  - Unfortunately **`@Contended`** can be used only in Java's libraries
- Non-blocking CAS-based algorithms can be very slow
  - Because threads perform updates by "busy wait"
  - And because of cache coherence overhead: M → I → M → I → M → …
- Non-blocking CAS-based algorithms can be very fast
  - Provided mutable data, incl. lock objects, are **mostly thread-local**
  - So data are core-local and hence cache-local and can stay in M state

References and materials

- Lecture notes chapter *8 The computer's CPU and RAM*
  - In particular sections 8.5 to 8.7 about multi-core and cache coherence
- Lecture notes chapter *9 Scalability case studies*
  - Section *9.1 Platform threads, virtual threads and futures*
  - Section *9.2 Long adder scalability case study*
  - Section *9.3 Concurrent hash map scalability case study* (cursory only)
- Goetz et al. sections 6.1, 6.2
- Next weeks:
  Week 10: Java data streams and functional programming
  Week 11: Programming Graphics Processing Units (GPUs)